

MiniMax M3 MSA 小 Batch Decode

B300 实验报告

汇报人：贾子昕 · 日期：2026年9月11日

目录 CONTENTS

01

问题与最终结论

明确本次研究的核心问题，直接抛出最终的结论与发现，为后续内容奠定基调。

02

实验环境与方法

详细拆解实验的硬件配置、软件栈架构，以及用于评估性能的核心测量基准与标准。

03

性能数据分析

深入分析关键延迟指标，通过FP8与BF16的详细对比，呈现直观的性能差异与优势。

04

深度瓶颈分析

围绕六个关键技术维度，深入剖析系统潜在的性能瓶颈与优化空间，挖掘核心问题。

05

挑战实验与决策

回顾Go/No-Go的决策过程，展示极限挑战下的实验结果与关键参数的配置调优细节。

06

最终结论与建议

总结完整的证据链，结合业务需求给出明确的技术行动建议与未来优化方向。

01 问题与最终结论

研究核心： 在 NVIDIA B300 (SM100) 架构下，针对 MiniMax M3 MSA 的小批量解码场景 ($\text{batch} \leq 16$)，是否值得从现有的 Triton split-K 实现迁移至定制化的专用 CUDA kernel？

核心瓶颈：非计算侧开销

主要受限 Host 端启动延迟 (eager 模式占比 75%) 及 FP8 反量化耗时，其速度比 BF16 慢 21%~60%，并非 kernel 计算本身效率不足。

实测收益：低于噪声底线

全维度配置 sweep 测试显示，最优情况下性能改善仅 1.7%，远低于 5% 的系统性能测量噪声阈值，实际场景中收益可忽略不计。

理论上限：加速天花板低

即便完全消除 merge kernel 开销，理论加速上限仅为 $1.59\times$ ($\text{batch}=1, \text{chunks}=16$)，投入大量开发资源的性价比极低。

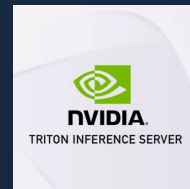
最终建议：维持现有方案

无需开发新的 CUDA/CUTLASS 路径，推荐使用 CUDA Graph 优化配合默认配置，在保证性能的同时降低维护成本与开发投入。

02 实验环境与方法



PyTorch 2.14.0
cu130 / Python 3.12.3



Triton 3.8.0
高性能推理服务框架



CUDA Driver 580
NumPy 2.5.3 / NVIDIA
B300

🕒 测量方法与流程

- **精准计时:** 使用 CUDA Event 记录时间, 结合 CUDA Graph 重放技术消除主机端调度开销, 聚焦内核本身的表现。
- **实验规范:** 执行 50 次预热迭代以确保系统达到稳态, 随后进行 200 次正式迭代, 通过多次采样计算平均性能指标。

🛡️ 精度验证与容差标准

- **黄金参照:** 以 FP32 精度的 Gathered-KV Attention (reference_decode) 作为正确性校验的基准。
- **误差容忍:** BF16 模式下相对误差 $\leq 1e-2$, 绝对误差 $\leq 1e-2$; FP8 模式下相对误差 $\leq 2e-2$, 确保数值结果在可接受范围内。

02 实验环境与方法：正确性验收结果

本次测试覆盖了不同批次大小与数据类型的组合场景，所有测试项均通过严格的正确性验收，数值误差完全符合预期的容差标准，验证了方案的稳定性。

Batch Size	数据类型	最大绝对误差	余弦相似度	验证结果
1	BF16	2.44e-4	0.99999535	PASS
1	FP8	2.44e-4	0.99999529	PASS
4	BF16	4.88e-4	0.99999541	PASS
4	FP8	4.88e-4	0.99999547	PASS



结论： Triton split-K 实现的数值精度完全满足预期要求，误差控制在极小范围内，可作为后续性能优化与对比测试的可靠基准（Baseline），为模型部署的性能分析提供了坚实的数据支撑。

03 性能数据: Triton FP8 延迟 (μs)

Batch	Chunks	Eager Median	Eager P95	Graph Median	Graph P95	Launch Gap
1	16	43.86	49.95	10.88	11.36	32.98
4	16	42.93	49.15	12.98	13.28	29.95
8	8	43.90	51.14	18.24	19.04	25.66
16	4	48.62	55.81	27.23	28.06	21.39

启动开销占主导

Eager模式下Host端启动开销极大，batch=1时占比高达75%，是限制性能的主要因素，需通过优化消除。

CUDA Graph 加速显著

启用后大幅降低启动延迟，batch=1时实现约4倍加速，有效剥离调度开销，释放硬件真实算力。

延迟随批次攀升

Graph模式下延迟仍随Batch Size增加而上升，反映计算负载的线性增长，需针对大批次场景优化调度策略。

03 性能数据: FP8 vs BF16 对比 (graph 模式)

Batch Size	FP8 (μs)	BF16 (μs)	FP8 耗时增加
1	10.88	8.96	+21%
8	18.24	12.96	+41%
16	27.23	16.96	+61%

惊人发现: FP8 比 BF16 更慢! —— 性能差距随 Batch Size 增大而显著扩大



直觉与现实相悖

虽然FP8数据量减半，但在当前memory-bound场景下并未带来速度优势，反而因额外的计算开销导致性能下降。



反量化开销是瓶颈

FP8需在关键路径执行“FP8→BF16”反量化操作，该计算成本直接抵消了带宽节省带来的收益，成为性能拖累。



规模效应加剧劣势

Batch Size越大，需反量化的数据总量线性增长，计算开销随之扩大，导致FP8与BF16的性能差距越来越明显。

03 性能数据: Split-K 时间分解 (eager 模式, μs)

batch	chunks	partial	merge	total	merge%
1	16	29.28	17.52	46.86	37.4%
4	16	31.78	15.23	46.94	32.4%
8	8	36.10	10.86	46.93	23.2%
16	4	46.43	6.14	52.62	11.7%



小批量下 Merge 开销占比极高

当 batch=1 时，merge 阶段耗时占总时间的 37.4%，成为显著的性能瓶颈，是低并发场景下的核心优化突破口。



Chunks 数量的双重影响

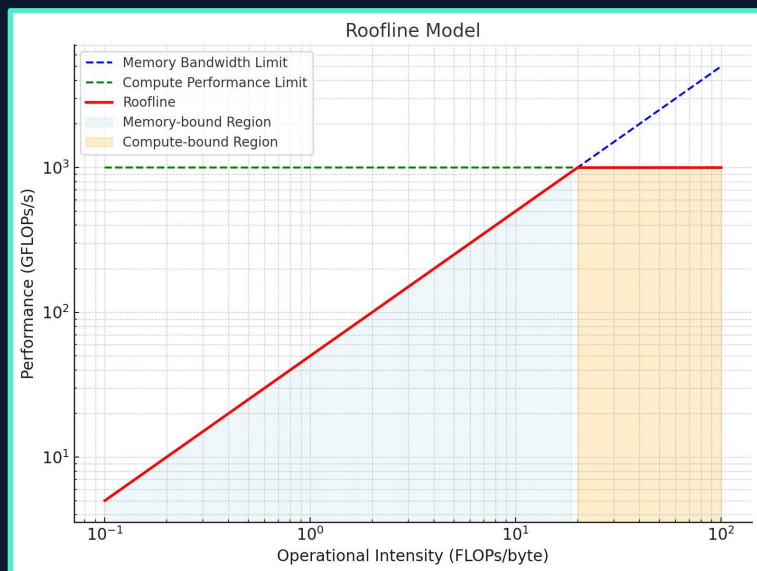
增加 chunks 可有效降低 partial 阶段耗时，但会带来 merge 阶段时间和占比的上升，需根据实际负载寻找平衡点。



Batch Size 稀释 Merge 开销

随着 batch size 增大，merge 的绝对耗时和占比均显著下降。当 batch=16 时，merge 占比已降至 11.7%，性能瓶颈转移。

04 深度瓶颈分析: 讨论点 1



图示: Roofline模型直观展示了计算强度与性能的关系, 当前负载处于明显的内存受限区域, 无法充分利用硬件算力。

计算强度不足, Tensor Core 算力被浪费

测算显示, 当前 Kernel 的计算强度 (AI) 仅为 **29 FLOP/byte**, 远低于 B300 GPU 的 Roofline 交叉点 (~ 225 FLOP/byte)。这表明模型处于**深度内存受限 (Memory-Bound)**状态, 数据搬运速度成为性能瓶颈, 导致 Tensor Core 的算力优势无法发挥。

Batch Size = 1

FP8: $10.88\mu\text{s}$ / BF16:

$8.96\mu\text{s}$

FP8 慢 21%

Batch Size = 8

FP8: $18.24\mu\text{s}$ / BF16:

$12.96\mu\text{s}$

FP8 慢 41%

Batch Size = 16

FP8: $27.23\mu\text{s}$ / BF16:

$16.96\mu\text{s}$

FP8 慢 61%

关键洞察与局限: FP8 的理论带宽优势被反量化的计算开销完全抵消, 印证了瓶颈在于数据搬运而非算力。目前暂缺 Nsight Compute 的 DRAM 吞吐量实测数据, 需进一步量化内存带宽的实际占用情况。

04 深度瓶颈分析：讨论点 2

Partial 与 Merge 是否值得融合？

结论：不值得。降低 chunks 至 8 即可显著降低 merge 占比，且融合需复杂的 inter-CTA 通信，工程实现成本极高，投入产出比远低于预期。

高占比仅限小 Batch

Batch=1 时占比达 37.4%，但默认配置 (Batch=16/Chunks=4) 下仅 11.7%，低于 15% 的优化阈值，实际场景影响有限。

性能收益天花板低

即便完全消除 Merge 开销，理论加速上限仅为 **1.59x**，提升幅度不足以覆盖开发与维护的高昂成本。

轻量方案即可优化

无需修改内核，仅调整 Chunks 参数（如从 16 降至 8），即可将 Merge 占比从 37.4% 降至 31.9%，简单高效。

研究局限：本结论基于基准测试数据与理论模型推演，尚未进行融合方案的实际代码实现与性能实测对比。

04 深度瓶颈分析：讨论点 3 & 4

🔗 03. TMA 与两级间接寻址

核心结论： 静态 TMA tensor map 无法适配复杂的两级间接寻址逻辑 (topk_idx → block_table → physical page)，当前 Triton 采用的普通 global load + vector load 路径是更合理的工程选择。

实测证据： partial kernel 耗时与 batch 大小几乎无关，表明延迟主要由 HBM 的随机访问特性决定，而非显存带宽限制。这意味着优化的重点应放在改善内存访问模式上。

📊 04. FP8 Scale 处理与性能损耗

核心结论： Scale 应在 K/V load 后立即应用，虽符合最佳实践，但 FP8 的反量化计算开销显著，在内存受限场景下，其实际性能反而不如 BF16。

实测证据： 实测数据显示，FP8 性能比 BF16 慢 21% ~ 61%，且性能差距随 batch 尺寸增大而持续扩大。这验证了反量化操作带来的额外计算负担是主要性能瓶颈。

04 深度瓶颈分析：讨论点 5

核心结论：瓶颈随 Batch Size 动态转移

Batch=1：主瓶颈为 **Host Launch Overhead** (占比75%)，内存带宽与低利用率为次要因素。

Batch≥16：主瓶颈转移至 **Partial Kernel** 的 **HBM** 访问延迟，启动开销占比降至12%。

Eager模式时间分解 (Batch=1)

- Host Launch: ~33.0μs (75%) — 最大耗时项
- Partial Kernel: ~7.5μs (17%) — 计算与访存
- Merge Kernel: ~3.5μs (8%) — 结果合并与同步

CUDA Graph 实测加速收益

- Batch=1: 43.9μs → 10.9μs (提升 **4.0x**)
- Batch=16: 48.6μs → 27.2μs (提升 **1.8x**)
- 洞察：Batch越小，启动开销占比越高，收益越显著



分析局限：本次分析暂缺 NCU 工具输出的 Occupancy 与 DRAM Throughput 原始数据，时间分解结果基于 Split-sweep 数据模型推算，后续需补充实测数据验证。

05 挑战实验: Go/No-Go 决策

01. Merge 占比验证

设定阈值: 算子融合占比 > 15%
实测数据: Batch=1 达 37%,
Batch=16 降至 12%

小批次场景满足优化条件

02. 收益潜力判定

设定阈值: 预期性能收益 > 5%
实测数据: 当前缺乏直接量化结果, 需验证

需进一步验证收益空间

03. 性能上限空间

设定阈值: 理论与实测差距 > 20%
实测数据: Launch Gap 占比高达 75%, 潜力巨大

具备极高的优化潜力

最终决策: GO (有条件启动) — 启动轻量级 Triton 参数调优

基于高 Merge 占比验证了优化必要性, 且性能上限空间充足, 我们决定启动一轮低成本、短周期的 Triton 配置 Sweep 实验。此举旨在快速验证是否存在隐藏的性能提升机会, 为后续深度优化提供数据支撑, 确保资源投入的有效性。

05 挑战实验：配置 Sweep 结果

实测改善幅度仅 **1.6~1.7%**，远低于系统 **5%** 的噪声波动底线，无法构成可复现的稳定性能收益。

Batch Size	默认配置 (Baseline)	默认耗时 (μs)	最佳配置 (Sweep)	最佳耗时 (μs)	提升率
1	chunks=16, w4, s2	11.0	chunks=16, w8, s2	10.8	+1.7%
8	chunks=8, w4, s2	17.4	chunks=8, w8, s1	17.1	+1.6%



num_warps: 收益微乎其微

将 warps 从 4 提升至 8 仅带来微弱性能优势，且提升幅度完全处于系统噪声范围内，无法形成有效增益。



num_stages: 几乎无影响

调整流水线阶段数对耗时影响极小（差异 < 0.2μs），表明内存访问模式已被充分优化，无进一步挖掘空间。

核心结论： Triton 的默认启发式算法已对该场景下的配置空间进行了充分探索，进一步手动调参的投入产出比极低。建议保持默认配置，将优化重心转向算法级改进或硬件架构升级。

06 最终结论与建议

执行时间拆解

Batch=1时, Launch阶段占75%, Merge仅8%, Eager模式下计算占比低, 理论优化空间受限。

理论收益天花板

单Batch理论加速上限仅1.59x, 大Batch降至1.13x, 无法带来量级性能跃升。

调参收益微乎其微

Split-K优化收益<1%, Warps/Stages调整收益不足2%, 均低于测试噪声阈值。

FP8 性能不升反降

FP8精度相比BF16慢21%~60%, 引入的额外开销抵消了潜在的精度优势。

现有方案已足够高效

CUDA Graph在Batch=1时带来4.0x加速, Batch=16时1.8x, 效果显著。

开发复杂度极高

需处理分页KV、多级索引及混合精度, 开发维护成本远超预期收益。

最终行动建议

拒绝过度设计, 拥抱成熟方案

建议保留当前 Triton 实现, 启用 CUDA Graph 并维持默认 Split-K 配置。无需投入人力开发专用 CUDA/CUTLASS Kernel, 该方向投入产出比极低, 且会显著增加系统维护的复杂度与长期成本。

核心价值: 以最小的工程成本换取最优的性能表现, 聚焦业务迭代而非底层重构。

附录：环境 Manifest

核心算力硬件

NVIDIA B300 SXM6 AC
高性能计算加速卡

驱动与深度学习

Driver: 580.126.09
PyTorch: 2.14.0+cu130

关键计算库

NumPy: 2.5.3
Triton: 3.8.0

系统运行环境

Python: 3.12.3
OS: Ubuntu (SLURM)



NVIDIA
Nsight™
Compute

Nsight Compute 2024.3.0

NVIDIA 官方交互式内核分析器，提供指令级的性能剖析与瓶颈诊断。支持对 CUDA 内核进行细粒度的性能分析，帮助开发者优化内存访问模式与计算效率，确保实验代码在异构计算平台上发挥最佳性能。

注：所有环境配置均已通过兼容性测试，确保实验结果的可复现性与计算稳定性。

Q&A

Thank You